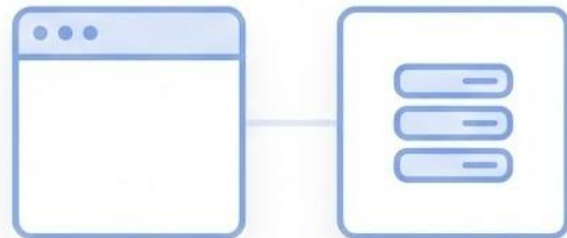


Next.js – Основи и Full-Stack Подход

Как работи Next.js и как се използва за създаване на реални full-stack уеб приложения



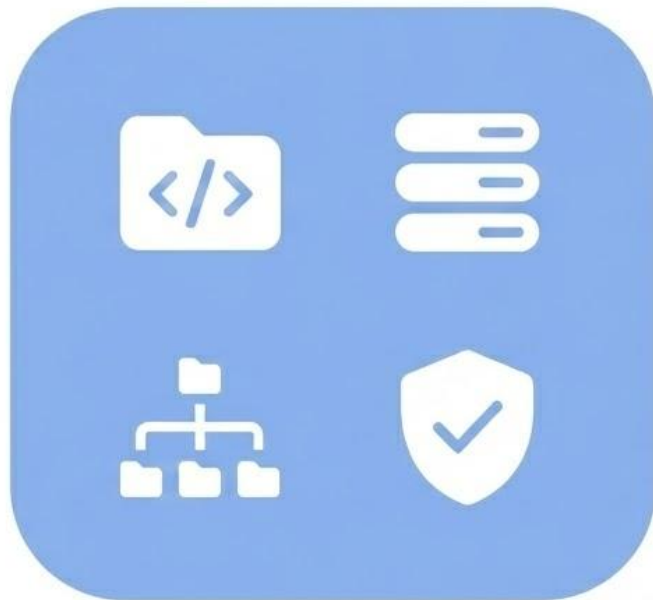
Какво е Next.js

- React framework за създаване на уеб приложения
- Комбинира frontend и backend логика в един проект
- Предоставя готови решения за routing и server-side функционалност



Защо Next.js

- Вграден routing без допълнителна конфигурация
- Backend логика в рамките на същия проект
- Ясна и стандартизирана структура
- Подходящ за реални production приложения



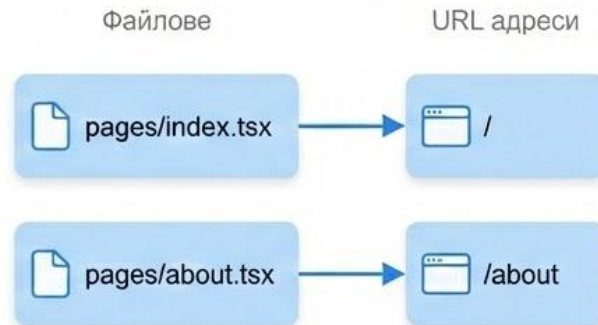
Основни концепции

- File-based routing
- Pages Router и App Router
- API routes
- Клиентска и сървърна логика



Routing в Next.js

- Всеки файл представлява маршрут
- URL адресът се определя от името на файла
- Не се пише ръчна router конфигурация



Pages Router и App Router

- Два подхода за routing в Next.js
- Pages Router – по-опростен и директен модел
- App Router – по-модерен и по-гъвкав подход



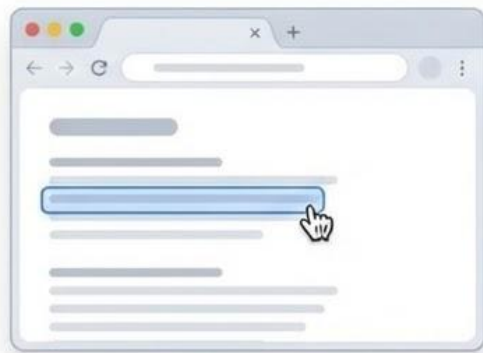
Структура на Next.js приложение

- pages/ или app/ - routing и страници
- api/ – backend логика
- Компоненти и конфигурация

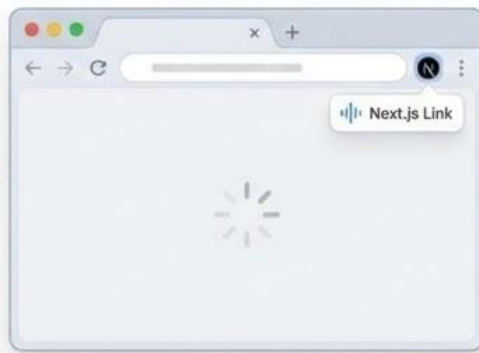


Навигация в Next.js

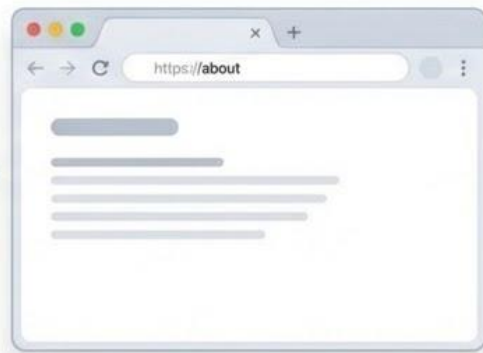
- Клиентска навигация чрез `Link`
- Без пълно презареждане на страницата
- Поведение, подобно на Single Page Application



Текуща страница



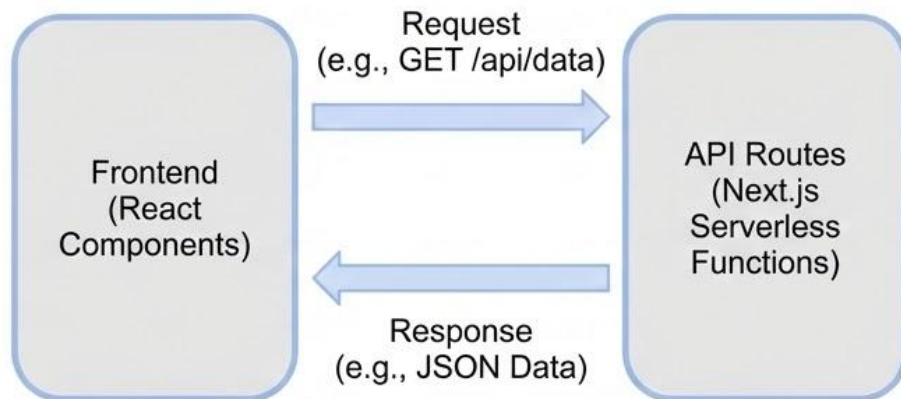
Преход (Client-side)



Нова страница

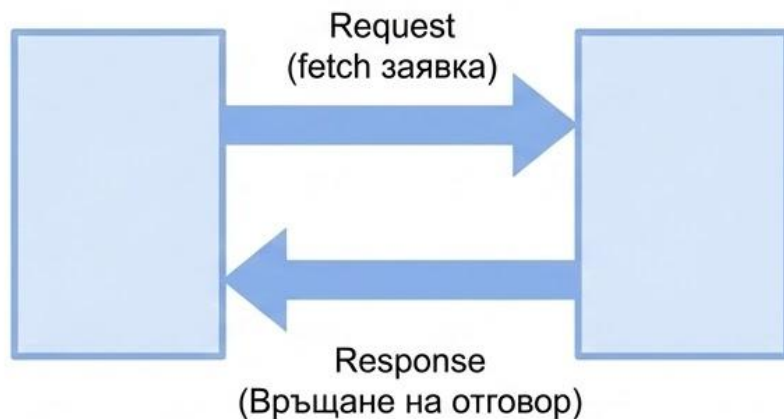
API Routes

- Backend endpoints в рамките на Next.js проекта
- Обработка на HTTP заявки (GET, POST и др.)
- Не е необходим отделен backend сървър



Frontend ↔ Backend комуникация

- Потребителско действие във frontend
- fetch заявка към API route
- Обработка на заявката на сървъра
- Връщане на отговор към клиента



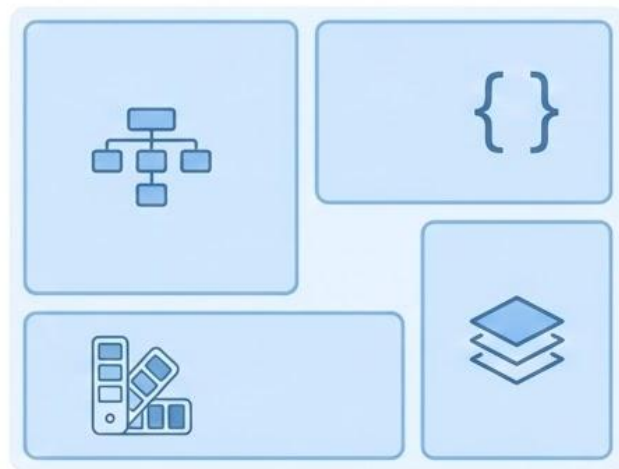
Какво включва едно full-stack приложение

- Frontend (потребителски интерфейс)
- Backend логика и API
- Данни (база данни или външни услуги)
- Автентикация и сигурност



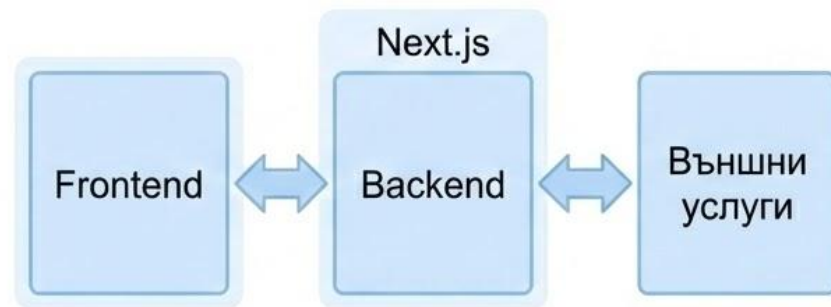
Frontend технологии в Next.js

- React компоненты
- TypeScript
- Styling (CSS, CSS Modules)
- Utility-first подход (напр. Tailwind CSS)



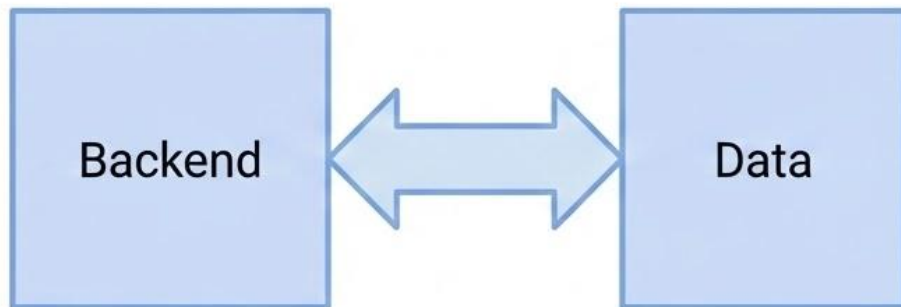
Backend логика и услуги

- API Routes като backend слой
- Обработка на заявки и бизнес логика
- Връзка с бази данни и външни услуги



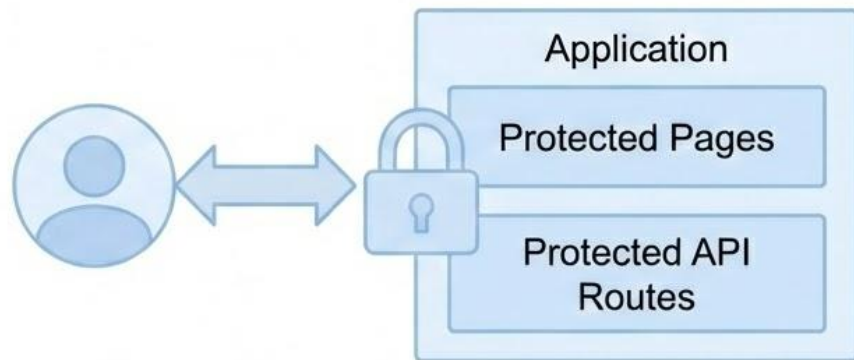
Работа с данни

- Четене и запис на данни
- Работа с база данни или API
- Отделяне на data layer от UI



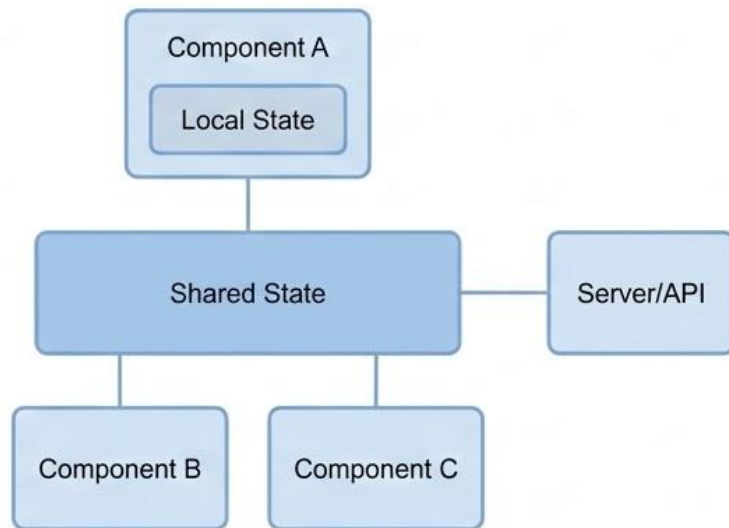
Автентикация и потребители

- Вход и регистрация
- Защитени страници и API маршрути
- Управление на потребителски сесии



Управление на състояние

- Локално състояние в компоненти
- Споделено състояние между страници
- Данни от сървъра и асинхронни заявки



Rendering стратегии

- Client-side rendering
- Server-side rendering
- Static generation
- Комбиниране на подходи

Client-side rendering (CSR)



Server-side rendering (SSR)



Static generation (SSG)



Архитектурен поглед

- Разделяне на отговорностите
- Ясни граници между слоевете
- Подготовка за надграждане

Presentation Layer
(UI, Components)

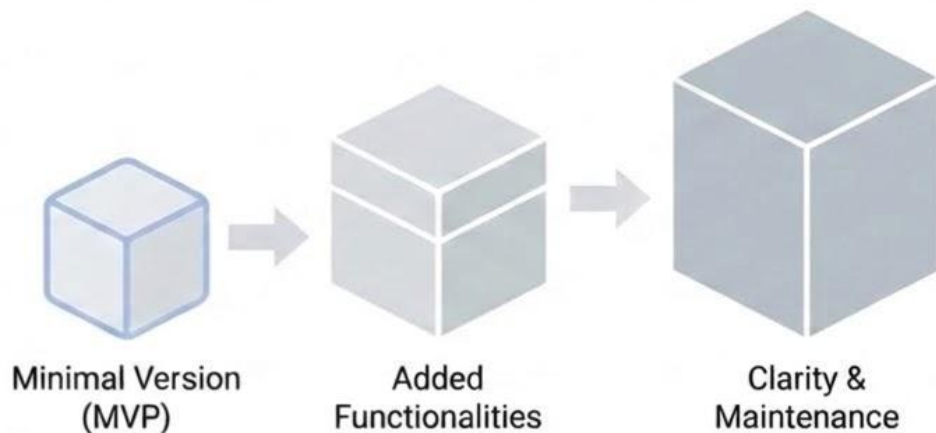
Application Layer
(Business Logic, API Routes)

Data Access Layer
(ORM, DB Clients)

Infrastructure Layer
(Database, External Services)

Подход към реален проект

- Започване с минимална версия
- Постепенно добавяне на функционалности
- Фокус върху яснота и поддръжка



Накъде оттук

- Самостоятелна работа по проект
- Изследване на Next.js екосистемата
- Усъвършенстване чрез практика

